

DOCUMENT 4 — QUALIFYAI SOFTWARE ARCHITECTURE & DEVELOPMENT STRUCTURE

Document Type: Software Architecture & Development Structure

Project: QualifyAI

Purpose: Complete codebase architecture, application layers, API flow, AI/voice processing, security, data flow, and development organization.

Document Purpose

This document defines how the **QualifyAI** application should be organized at the software architecture and engineering level.

It translates the Product & Functional Blueprint, Technical Implementation Plan, and Frontend Blueprint into a practical development structure that developers can directly implement.

The architecture is designed around the separation of:

- Presentation
- API handling
- Business logic
- AI orchestration
- Real-time communication
- Evaluation
- Proctoring
- Data access
- Storage
- Authentication and authorization

The reference architecture follows the same principle of separating interface, API handling, business services, AI processing, and persistent data rather than mixing these responsibilities together. Document 4

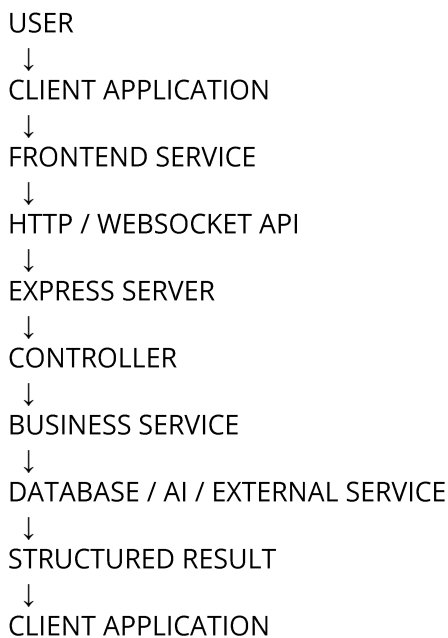
1. Architecture Philosophy

QualifyAI should be structured as a modular, scalable SaaS application while remaining practical for rapid development.

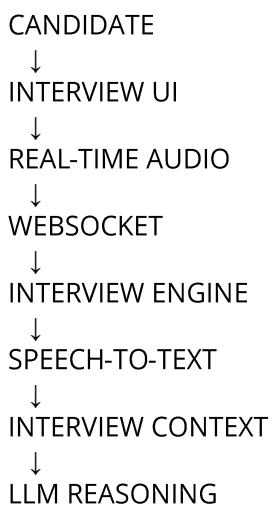
The most important architectural principle is:

Separate presentation, API handling, business logic, AI processing, real-time communication, evaluation, and data access.

The basic application flow is:



AI-driven interview workflows extend this architecture:



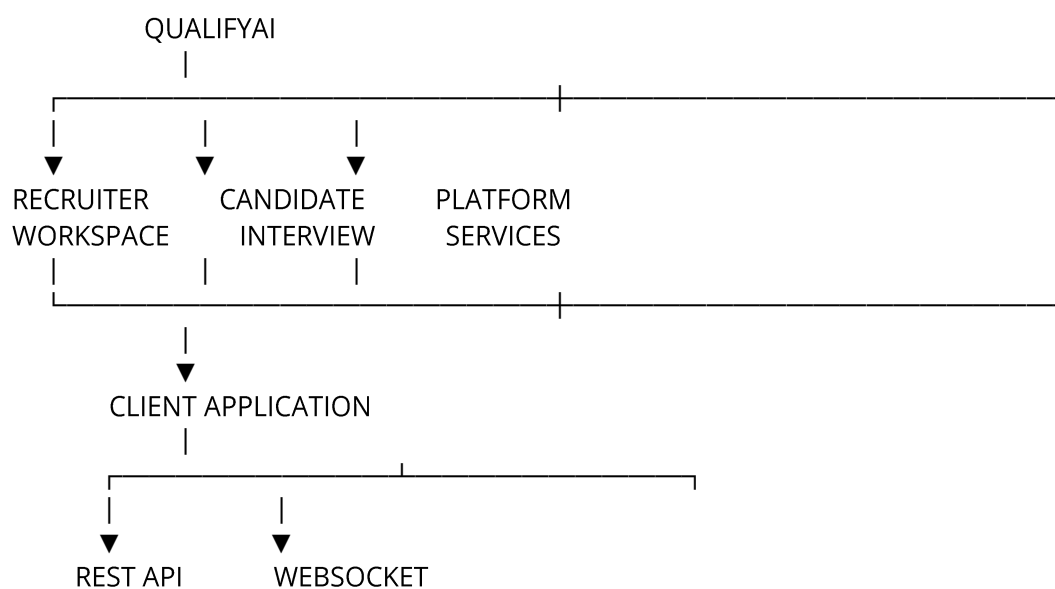
↓
NEXT QUESTION / FOLLOW-UP
↓
TEXT-TO-SPEECH
↓
CANDIDATE

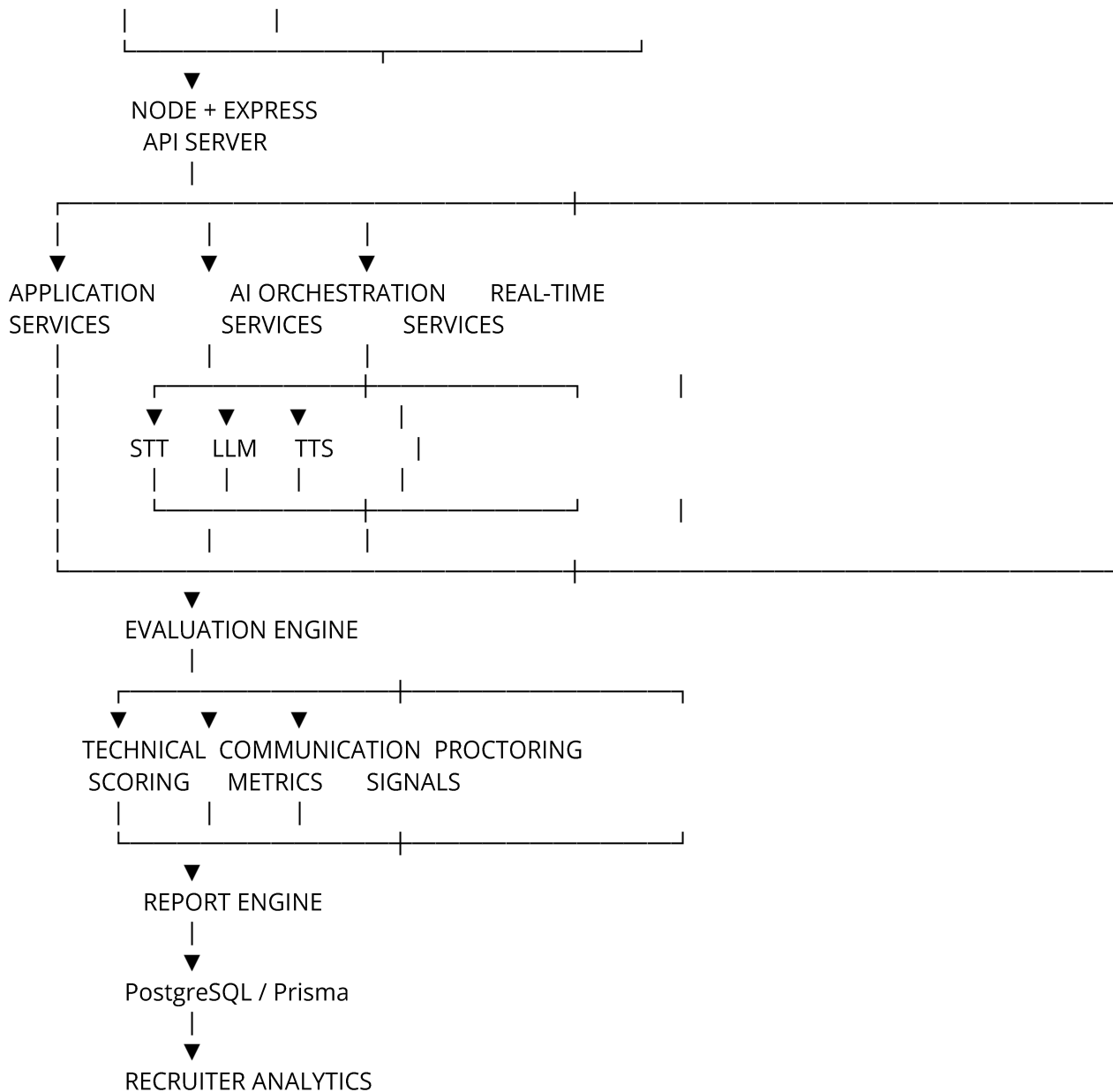
Evaluation runs alongside the interview workflow:

INTERVIEW TRANSCRIPT
↓
EVALUATION ENGINE
↓
TECHNICAL ANALYSIS
+
COMMUNICATION ANALYSIS
+
RUBRIC COMPARISON
+
PROCTORING SIGNALS
↓
SCORE
↓
REPORT
↓
RECRUITER DASHBOARD

2. Complete QualifyAI System Architecture

The overall architecture can be represented as:





3. Repository Structure

QualifyAI should use a monorepo-style structure while keeping the major application responsibilities clearly separated.

The required high-level structure is:

```
QUALIFYAI/  
|  
|—— README.md  
|—— prd.md
```

```
|— architecture.md
|— design.md
|— memory.md
|— phases.md
|— rules.md
|
|— client/
|
|— server/
|
|— storage/
|
|— document/
|
|— docs/
```

Responsibility of each root directory

Directory	Responsibility
client/	React/Next.js frontend application
server/	Node.js + Express backend
storage/	Logical storage organization for uploaded/generated artifacts
document/	Formal project documents and reference artifacts
docs/	Living technical and engineering documentation

The root documentation remains the permanent source of truth for the project.

4. Root Documentation Architecture

The seven root Markdown documents are:

```
QUALIFYAI/
|
|— README.md
|— prd.md
|— architecture.md
|— design.md
|— memory.md
|— phases.md
|— rules.md
```

README.md

Provides:

- Project overview
- Product purpose
- Major capabilities
- Technology overview
- Setup direction
- Repository overview

prd.md

Defines:

- Product requirements
- User roles
- User journeys
- Functional requirements
- Product scope
- Acceptance criteria

architecture.md

Defines:

- System architecture
- Application layers
- Data flow
- Integration architecture
- Security architecture
- Deployment architecture

design.md

Defines:

- Design system
- UI principles
- Interaction patterns
- Responsive behavior
- Visual hierarchy

memory.md

Maintains:

- Current implementation state
- Completed work
- Active tasks
- Important decisions
- Known issues
- Development history

phases.md

Defines:

- Implementation phases
- Dependencies
- Milestones
- Development sequence

rules.md

Defines:

- Coding rules
- Security rules
- Architectural constraints
- AI evaluation boundaries
- Data handling rules
- Multi-tenancy rules

5. Client Architecture

The client is responsible for the complete user-facing experience.

QualifyAI has three primary frontend experiences:

CLIENT

|

|—— Public Experience

|

|—— Recruiter Experience

|
└── Candidate Experience

The client is responsible for:

- UI rendering
- User interaction
- Form handling
- Navigation
- Authentication state
- API communication
- WebSocket communication
- Interview state visualization
- Audio interface
- Results visualization

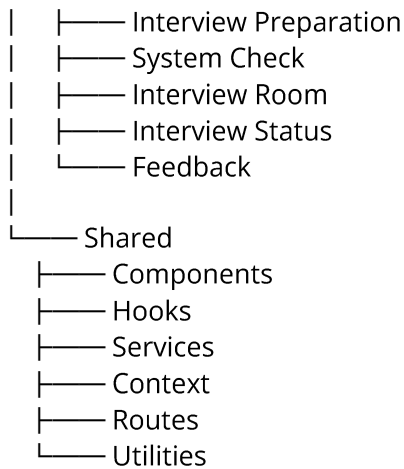
The client must **not** contain authoritative business logic.

6. Client Domain Architecture

The frontend should be organized around product domains rather than one large collection of unrelated components.

Conceptually:

```
CLIENT
|
├── Public
|
├── Authentication
|
├── Recruiter
|   ├── Dashboard
|   ├── Jobs
|   ├── Rubrics
|   ├── Questions
|   ├── Candidates
|   ├── Interviews
|   ├── Rankings
|   ├── Analytics
|   └── Reports
|
└── Candidate
```



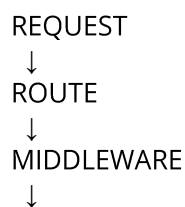
7. Server Architecture

The server is the primary application orchestration layer.

The server is responsible for:

- Authentication verification
- Authorization
- Organization isolation
- API routing
- Request validation
- Business logic
- Database access
- AI orchestration
- Interview session management
- Evaluation
- Proctoring
- Report generation
- Background processing
- External service communication

The basic server architecture is:



CONTROLLER
↓
SERVICE
↓
DATA / AI / EXTERNAL INTEGRATION
↓
RESPONSE

8. Server Module Architecture

The backend should be divided into clear functional domains.

SERVER

- | — Authentication
- | — Organizations
- | — Jobs
- | — JD Processing
- | — Rubrics
- | — Questions
- | — Candidates
- | — Invitations
- | — Interviews
- | — Interview Sessions
- | — AI Integration
- | — Evaluation
- | — Communication Metrics
- | — Proctoring
- | — Reports
- | — Analytics
- | — Notifications

└── Storage

Each domain should maintain clear responsibility boundaries.

9. API Architecture

QualifyAI uses APIs as the communication boundary between the client and server.

The basic request flow is:

```
CLIENT
↓
API SERVICE
↓
HTTP REQUEST
↓
EXPRESS ROUTE
↓
AUTH MIDDLEWARE
↓
VALIDATION
↓
CONTROLLER
↓
BUSINESS SERVICE
↓
DATABASE / AI / EXTERNAL SERVICE
↓
STRUCTURED RESPONSE
↓
CLIENT
```

Controllers should remain lightweight.

They should primarily:

1. Receive the request.
2. Validate required request context.
3. Invoke the appropriate service.
4. Return the structured response.

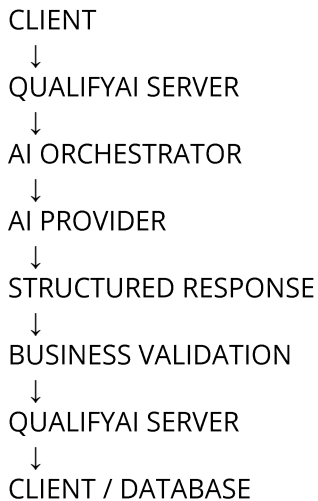
Business logic should remain inside services rather than controllers.

10. AI Integration Architecture

AI integrations are part of the server architecture.

QualifyAI should not expose AI provider credentials directly to the frontend.

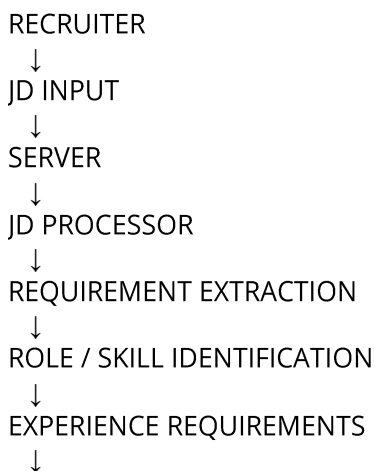
The architecture is:

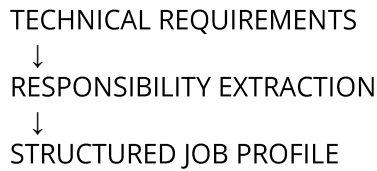


The AI integration layer should abstract external providers so that the rest of the application does not depend directly on provider-specific implementation details.

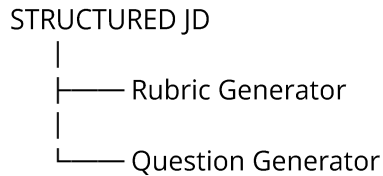
11. Job Description Processing Architecture

The JD processing pipeline begins when a recruiter creates or uploads a job description.



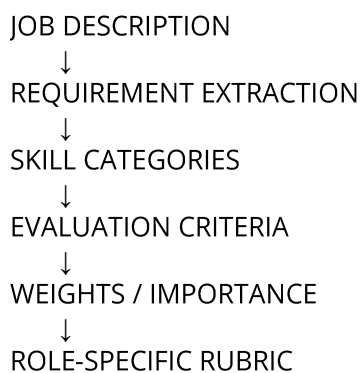


The structured job profile becomes the input for:



12. Rubric Architecture

The rubric is the evaluation framework generated from the job requirements.

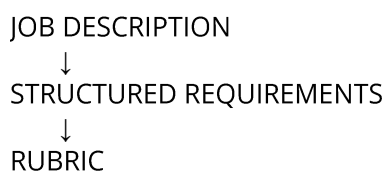


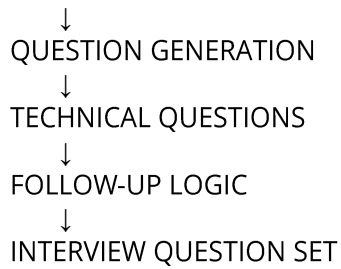
The rubric should remain associated with the job and interview.

It provides the evaluation engine with a structured basis for scoring candidates.

13. Question Generation Architecture

Question generation uses the role requirements and rubric as context.





Questions should be role-specific rather than generic.

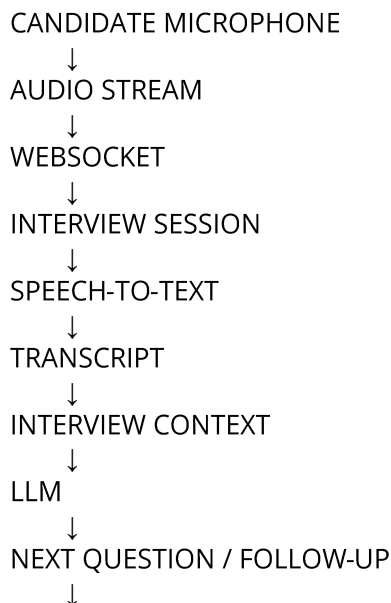
The question engine should support:

- Core technical questions
- Conceptual questions
- Problem-solving questions
- Role-specific questions
- Follow-up questions
- Clarification questions
- Difficulty adaptation

14. Real-Time Interview Architecture

The real-time interview is one of the core architectural components of QualifyAI.

The target flow is:



TEXT-TO-SPEECH
↓
AUDIO RESPONSE
↓
CANDIDATE

The documented implementation direction uses:

- WebSockets / Socket.io
- Deepgram for speech-to-text
- LLM processing
- ElevenLabs for text-to-speech

15. Interview Session Architecture

Every active interview should be represented as a controlled session.

Conceptually:

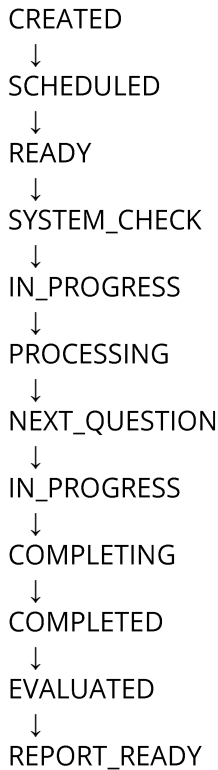
INTERVIEW
|
├── Candidate
├── Job
├── Rubric
├── Question Set
├── Session State
├── Transcript
├── Evaluation Data
├── Communication Metrics
├── Proctoring Events
└── Final Report

The interview session manages:

- Current question
- Candidate response
- Conversation context
- Follow-up decision
- Interview progress
- Connection state
- Completion state

16. Interview State Architecture

The interview should use explicit states.



Failure states should also be handled explicitly:

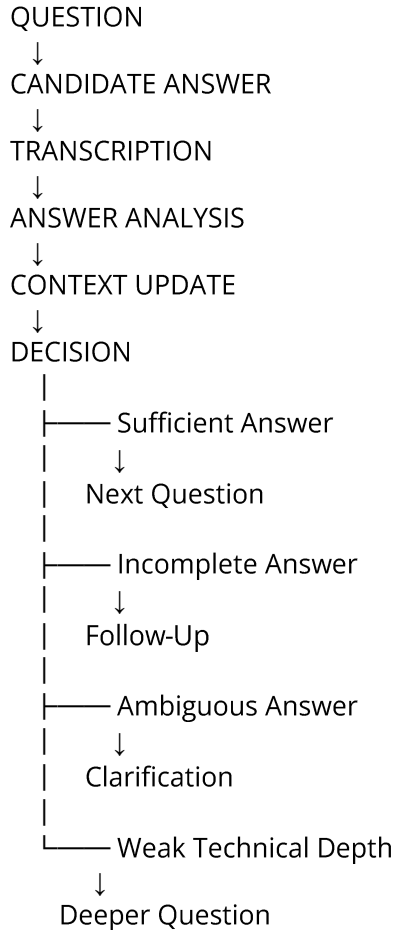
CONNECTION_ERROR
AI_ERROR
AUDIO_ERROR
TRANSCRIPTION_ERROR
SESSION_ERROR

The system should recover whenever possible rather than silently losing interview state.

17. Adaptive Interviewing Architecture

QualifyAI should not treat an interview as a static question list.

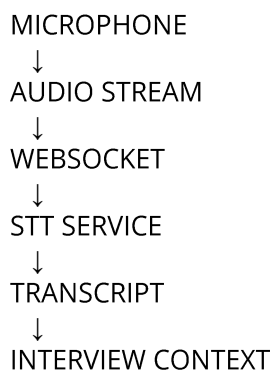
The adaptive loop is:



This creates the conversational behavior expected from the AI interviewer.

18. Speech-to-Text Architecture

Speech recognition is responsible for converting candidate audio into text.



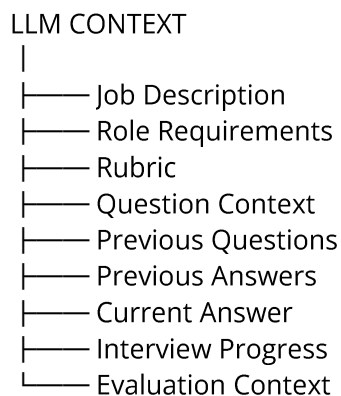
The transcript should preserve enough context for:

- Follow-up generation
- Evaluation
- Communication analysis
- Final reporting

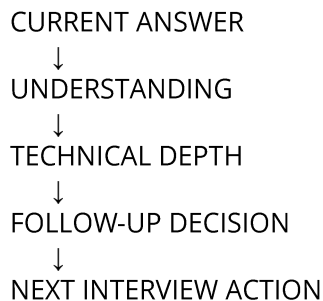
19. LLM Reasoning Architecture

The LLM receives structured interview context rather than an isolated candidate sentence.

Conceptually:



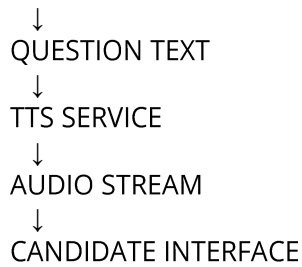
The LLM can then determine:



20. Text-to-Speech Architecture

After the interview engine generates the next question:

LLM OUTPUT



The target architecture aims for conversational interaction with approximately sub-1.5-second round-trip voice latency.

The documented target breakdown is approximately:

STT	→ < 300 ms
LLM	→ ~ 400 ms
TTS	→ ~ 350 ms

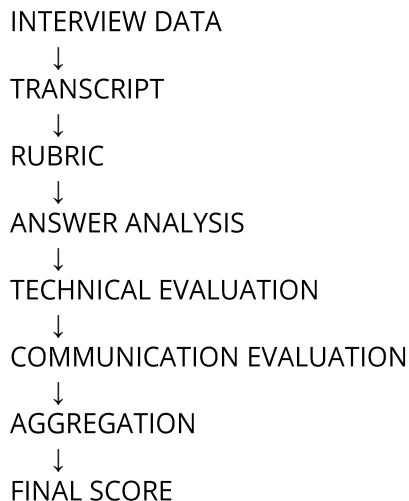
Target	→ < 1.5 sec

These are **engineering targets**, not guaranteed production measurements.

21. Evaluation Architecture

Evaluation should be separated from interview conversation generation.

The evaluation flow is:

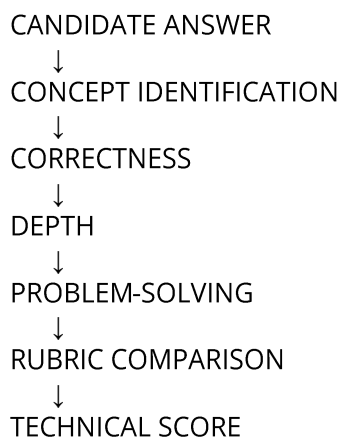


This separation prevents the conversational layer from becoming responsible for the complete scoring process.

22. Technical Evaluation Architecture

Technical evaluation considers the candidate's responses against the role-specific rubric.

Conceptually:



The evaluation should be explainable through evidence from the interview transcript.

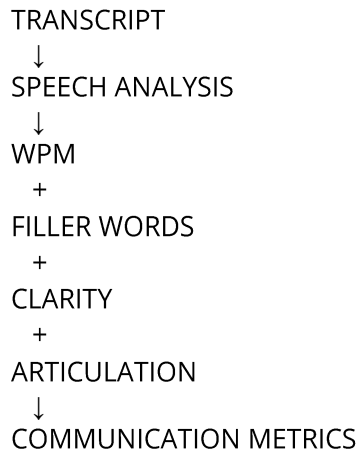
23. Communication Evaluation Architecture

Communication analysis operates on the candidate's spoken responses.

The documented metrics include:

- Communication clarity
- Speaking rate / WPM
- Filler-word density
- Response articulation

Conceptually:

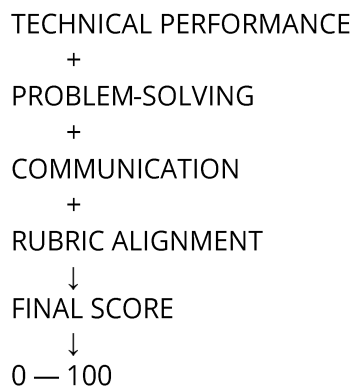


These metrics should complement technical evaluation rather than replace it.

24. Scoring Architecture

QualifyAI uses a structured **0–100 scoring model**.

The conceptual model is:



The exact weighting should remain configurable according to the defined product requirements rather than being hard-coded into the architecture.

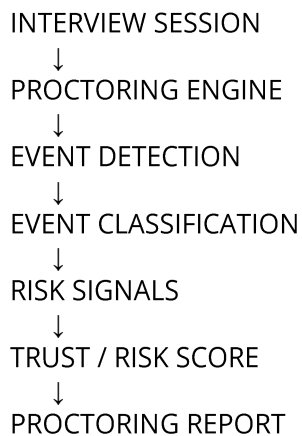
25. Proctoring Architecture

QualifyAI includes assessment-integrity mechanisms.

The proctoring layer can monitor:

- Browser visibility
- Focus loss
- Tab switching
- Acoustic anomalies
- Possible secondary voices
- Other defined interview integrity signals

The flow is:



26. Proctoring Boundaries

QualifyAI should **not** rely on facial-expression or micro-expression scoring as a candidate evaluation mechanism.

The core evaluation should remain focused on:

Technical Correctness

+

Technical Reasoning

+

Problem-Solving

+

Audio / Communication Metrics

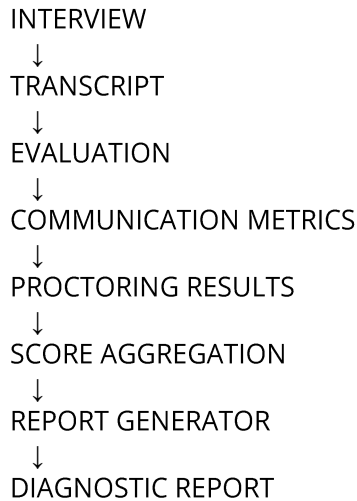
+

Assessment Integrity

This maintains a clearer boundary between technical assessment and potentially unreliable or biased visual interpretation.

27. Report Generation Architecture

After an interview is completed:

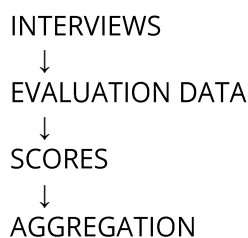


The candidate report can contain:

- Overall score
- Technical performance
- Communication metrics
- Strengths
- Improvement areas
- Interview summary
- Relevant evaluation evidence

28. Recruiter Analytics Architecture

Recruiter analytics consumes evaluated interview data.



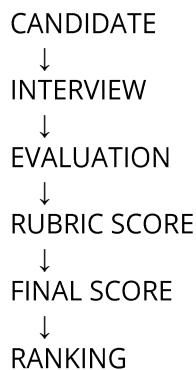


The recruiter should be able to inspect:

- Candidate scores
- Candidate ranking
- Interview status
- Technical performance
- Communication metrics
- Proctoring information
- Interview transcripts
- Generated reports

29. Candidate Ranking Architecture

Candidate ranking is generated from structured evaluation data.



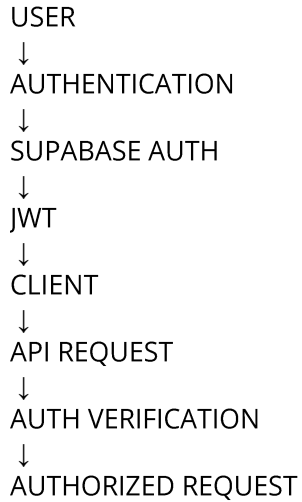
The ranking should be scoped to the relevant job/organization.

Candidates from one organization must never appear in another organization's ranking.

30. Authentication Architecture

Authentication is handled through the documented Supabase Auth/JWT direction.

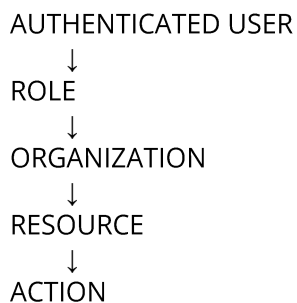
Conceptually:



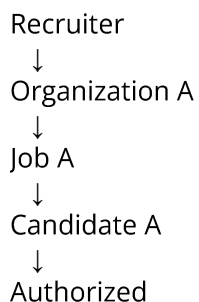
The server should never trust role or organization information supplied directly by an untrusted client.

31. Authorization Architecture

Authorization should be performed at multiple levels:



For example:

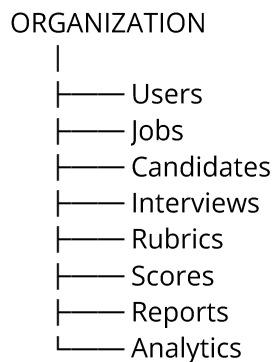


A recruiter associated with Organization A must not access Organization B's resources.

32. Multi-Tenant Architecture

QualifyAI is designed as a multi-tenant SaaS platform.

The core isolation principle is:



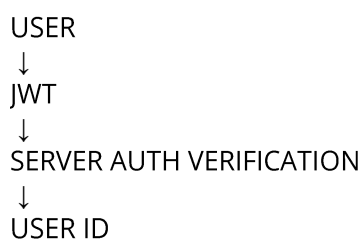
Every organization-owned resource should be associated with the appropriate organization context.

The documented architecture uses:

- PostgreSQL
- Prisma
- Supabase Auth/JWT
- Row-level security / tenant isolation principles

33. Multi-Tenant Data Flow

The secure data flow is:



↓
ORGANIZATION CONTEXT
↓
AUTHORIZED SERVICE
↓
DATABASE QUERY
↓
TENANT-SCOPED RESULT

Organization context should be established by trusted server-side authentication and authorization logic.

34. Database Architecture

The persistent data layer uses:

POSTGRESQL
↓
PRISMA
↓
SERVER SERVICES

Major conceptual entities include:

Organizations
Users
Jobs
Job Requirements
Rubrics
Questions
Candidates
Invitations
Interviews
Interview Sessions
Transcript Data
Scores
Communication Metrics
Proctoring Events
Reports

The final schema should be derived from the product requirements and implementation dependencies rather than creating unrelated entities.

35. Storage Architecture

The storage/ directory represents the logical storage boundary for generated and uploaded artifacts.

Potential categories include:

```
storage/  
|  
├── job-documents/  
├── interview-artifacts/  
├── transcripts/  
├── reports/  
└── temporary/
```

Actual storage-provider implementation should remain aligned with the project implementation plan.

Storage access must be tenant-aware.

36. Background Processing Architecture

Some tasks should not block interactive API requests.

The documented implementation direction includes:

```
APPLICATION SERVER  
↓  
JOB QUEUE  
↓  
REDIS  
↓  
WORKER  
↓  
BACKGROUND TASK
```

Potential asynchronous tasks include:

- Report generation
- Large-scale evaluation
- Analytics aggregation
- Notification processing

- Other non-interactive processing

BullMQ + Redis are the documented technologies for this layer.

37. Error Handling Architecture

Errors should be handled consistently across the system.

```
ERROR
↓
DOMAIN / SERVICE
↓
ERROR HANDLER
↓
STRUCTURED ERROR
↓
API RESPONSE
↓
CLIENT ERROR STATE
```

Errors should be categorized appropriately:

Validation Error
Authentication Error
Authorization Error
Resource Error
AI Error
Integration Error
Connection Error
Database Error
Processing Error

Sensitive internal details must not be exposed to end users.

38. AI Failure Handling

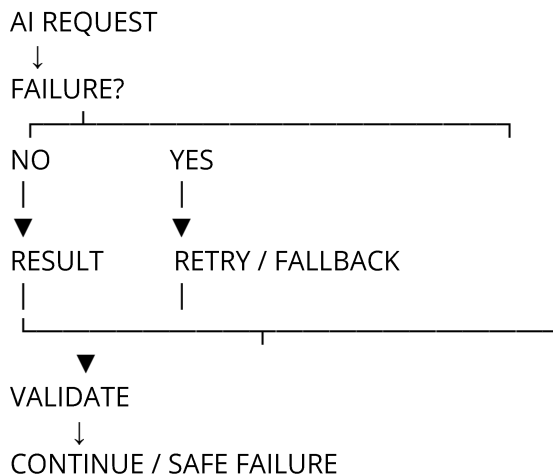
AI services are external dependencies and may fail.

The architecture must account for:

- Provider timeout

- Provider unavailability
- Invalid AI output
- Speech recognition failure
- TTS failure
- Network interruption
- Rate limiting

The preferred flow is:

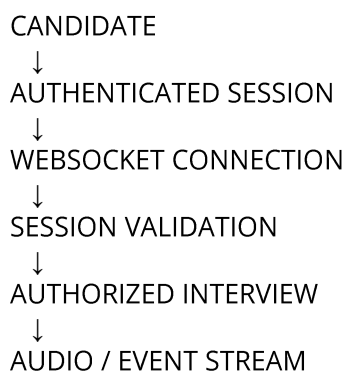


The system should never blindly trust unvalidated AI output.

39. Real-Time Security Architecture

WebSocket connections require authentication and session authorization.

Conceptually:



A candidate should only be able to interact with their own active interview session.

40. API Security

The server should enforce:

- Authentication
- Authorization
- Input validation
- Organization isolation
- Rate limiting where appropriate
- Secure secrets management
- Safe error responses
- Secure WebSocket sessions
- Controlled AI provider access

AI provider credentials must remain server-side.

41. Data Privacy Architecture

QualifyAI handles sensitive recruitment information, including:

- Candidate profiles
- Interview transcripts
- Evaluation results
- Communication metrics
- Proctoring events
- Generated reports

Therefore:

```
CLIENT
↓
SECURE API
↓
AUTHORIZED SERVER
↓
TENANT-SCOPED DATA
↓
DATABASE / STORAGE
```

Access should be limited according to role and organization.

42. Observability Architecture

The server should maintain structured operational visibility.

The architecture should allow tracking of:

REQUEST
↓
SERVICE
↓
AI CALL
↓
DATABASE
↓
RESULT

Important operational signals include:

- API latency
 - AI latency
 - STT latency
 - TTS latency
 - Interview connection status
 - Background-job status
 - Error frequency
 - Failed integrations
-

43. Performance Architecture

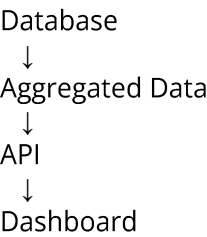
Performance-sensitive areas include:

Candidate Interview

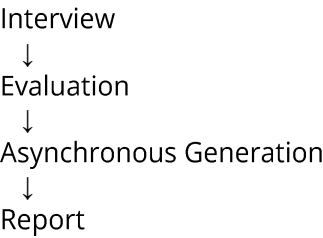
Audio Capture
↓
Streaming
↓
STT



Recruiter Dashboard



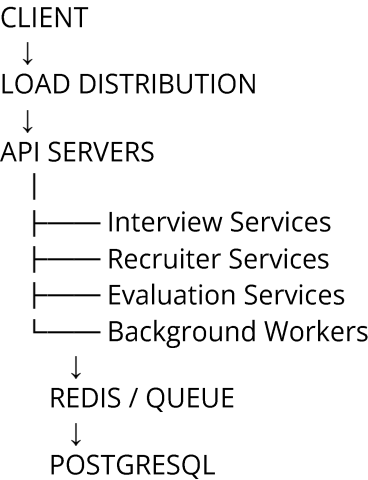
Reports



The interactive interview path should receive the highest latency optimization priority.

44. Scalability Architecture

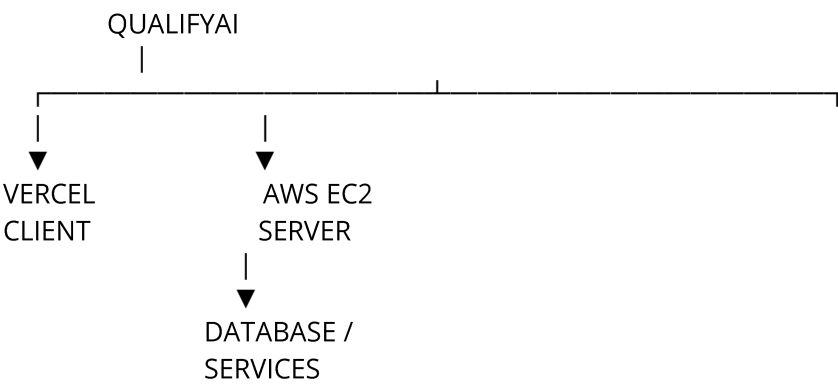
QualifyAI should be capable of scaling independently across major workloads.



The documented product target is support for **1,000+ simultaneous candidate assessment sessions**. This is a target architectural requirement, not a claim of already-achieved capacity.

45. Deployment Architecture

The documented deployment direction is:



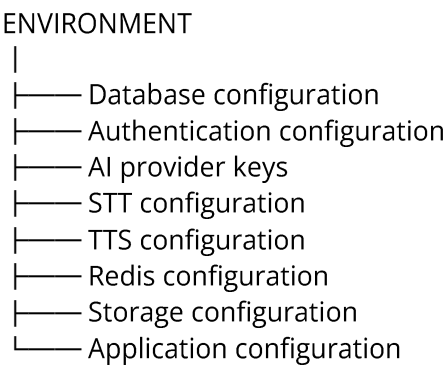
The implementation plan also identifies Docker as part of the deployment direction.

Deployment complexity should remain proportional to project requirements.

46. Environment Architecture

Environment-specific configuration should be separated from application code.

Conceptually:



Secrets must never be hard-coded into the repository.

47. Development Structure

Development should follow dependency order.

```
FOUNDATION
↓
AUTHENTICATION
↓
DATABASE
↓
JOB / JD PROCESSING
↓
RUBRIC
↓
QUESTION GENERATION
↓
INTERVIEW ENGINE
↓
STT / LLM / TTS
↓
EVALUATION
↓
PROCTORING
↓
REPORTS
↓
ANALYTICS
↓
OPTIMIZATION
↓
DEPLOYMENT
```

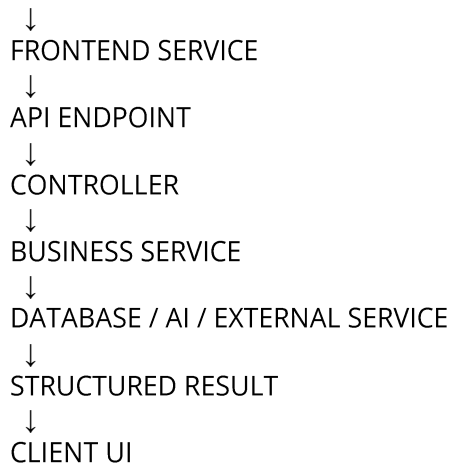
This sequence minimizes unnecessary rework.

48. Feature Development Traceability

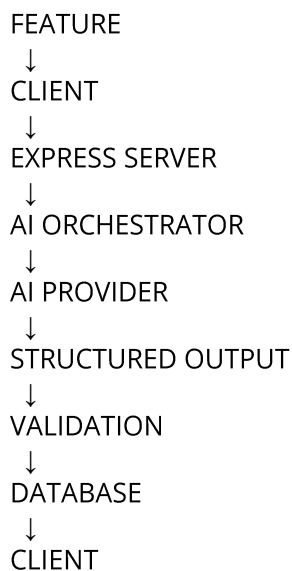
Every feature should be traceable through the architecture.

The expected pattern is:

```
FEATURE
↓
CLIENT EXPERIENCE
```



For an AI-heavy feature:



This follows the same traceability philosophy used by the reference architecture. Document 4(1)

49. Development Rules

The following rules should govern implementation:

Rule 1 — Separation of Responsibilities

Frontend components should not contain backend business logic.

Rule 2 — Controller Simplicity

Controllers should delegate business operations to services.

Rule 3 — AI Abstraction

External AI providers should be accessed through controlled integration layers.

Rule 4 — Tenant Isolation

Every organization-owned resource must remain organization-scoped.

Rule 5 — Server-Side Secrets

API keys and provider credentials must never be exposed to the client.

Rule 6 — Structured AI Output

AI-generated output must be validated before becoming authoritative application data.

Rule 7 — Explicit Interview States

Real-time interview behavior must use defined states rather than uncontrolled UI conditions.

Rule 8 — Evaluation Independence

Interview conversation generation and final evaluation should remain logically separated.

Rule 9 — Proctoring Boundaries

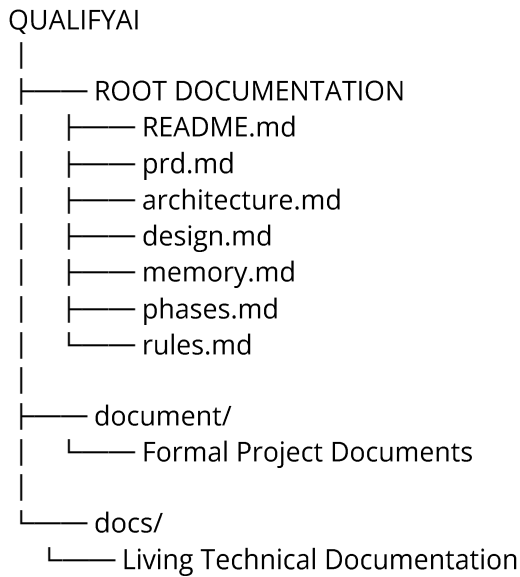
Proctoring should provide assessment-integrity signals and should not become an unsupported personality or facial-expression scoring system.

Rule 10 — Documentation Traceability

Major architectural changes must be reflected in the appropriate project documentation.

50. Documentation Architecture

The project documentation is divided into three layers.



document/

Contains formal project artifacts such as:

- Project documentation
- Architecture references
- Academic/project deliverables
- Approved specifications

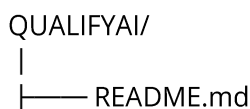
docs/

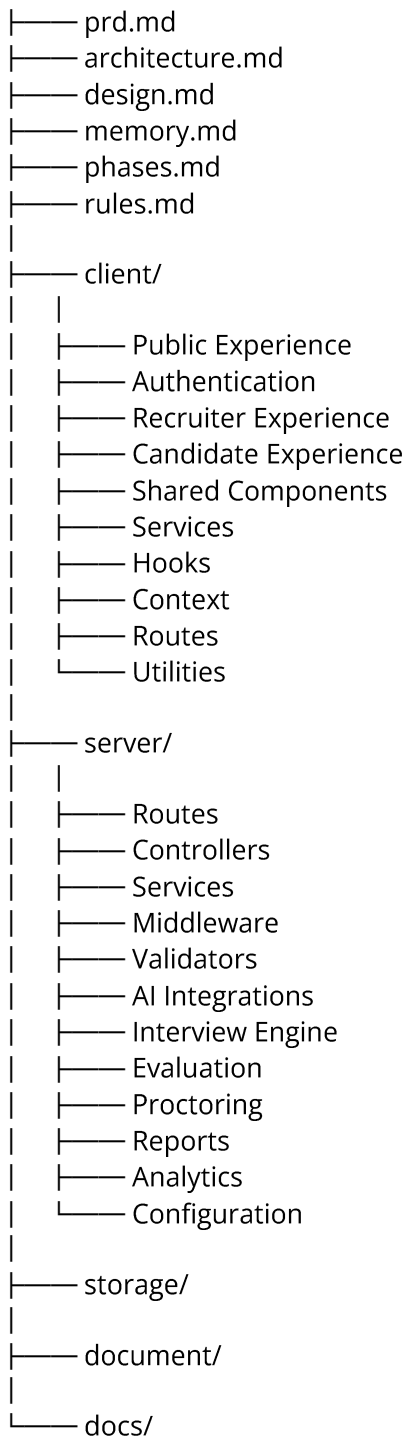
Contains continuously evolving engineering documentation such as:

- API specifications
- Integration notes
- Development guides
- Technical decisions
- Implementation references

51. Recommended Codebase Map

The high-level implementation map is:

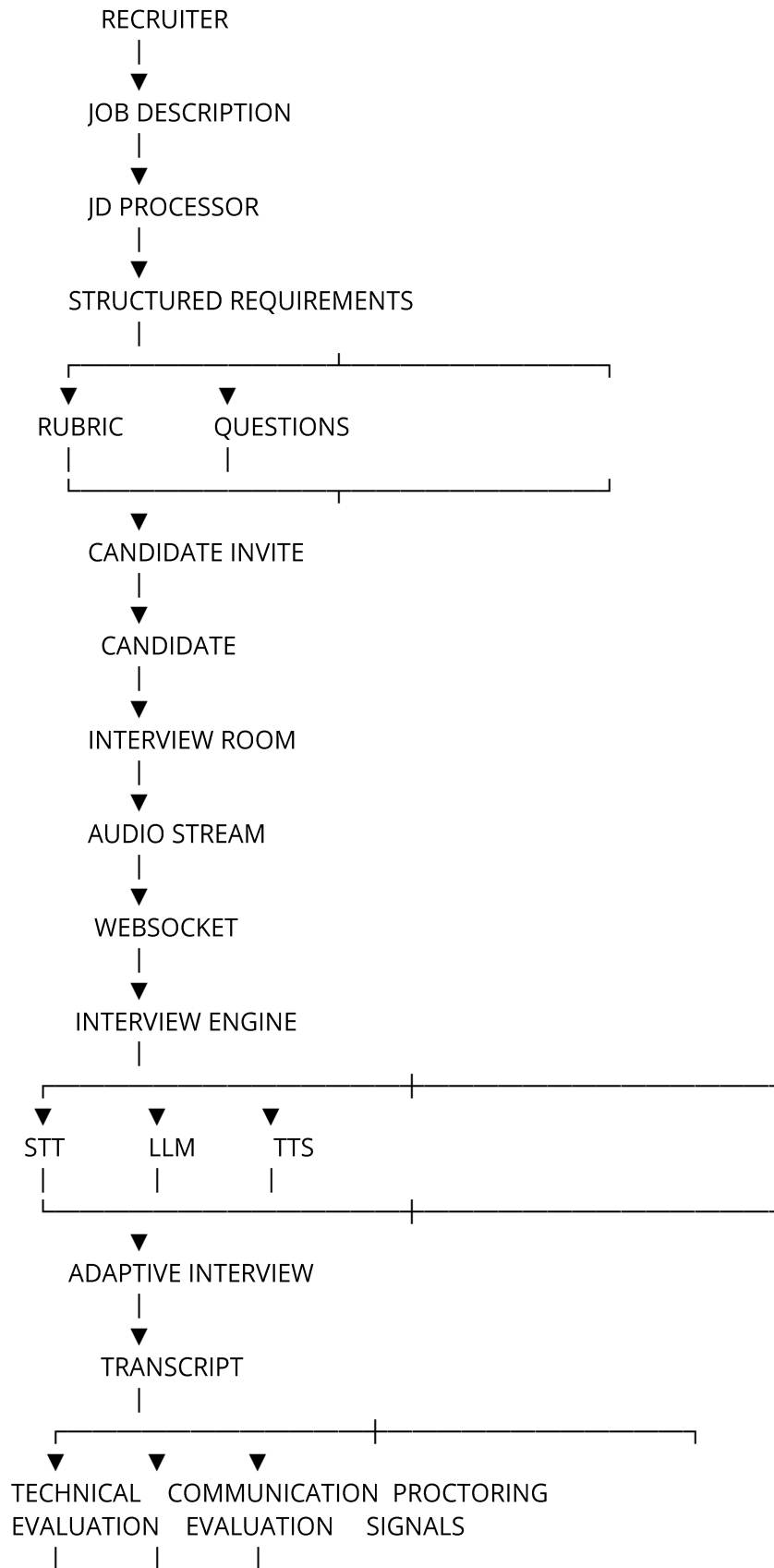


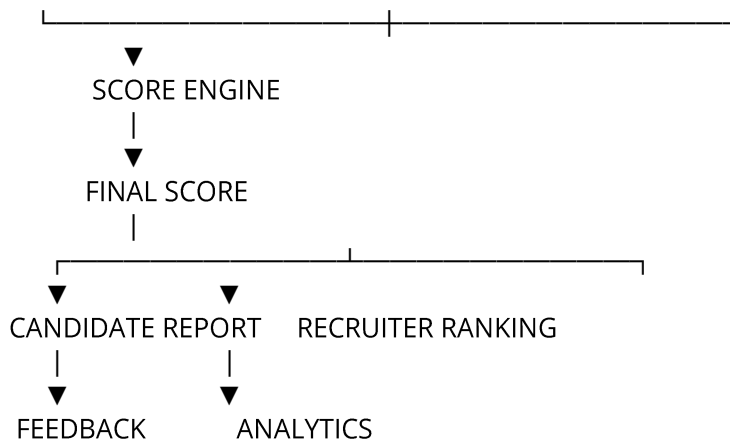


This deliberately keeps the structure conceptual at this level rather than inventing individual implementation files.

52. Complete End-to-End Architecture

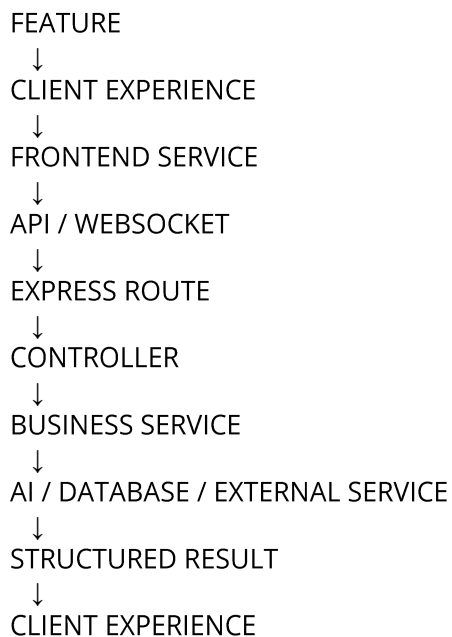
The complete QualifyAI workflow is:



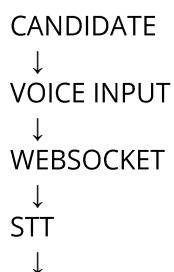


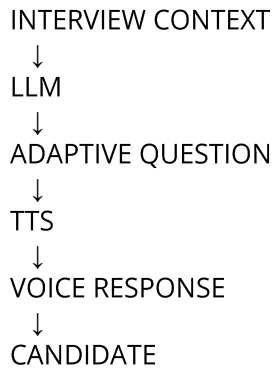
53. Final Architecture Principle

The complete QualifyAI architecture should remain traceable from the user-facing feature to the underlying implementation:

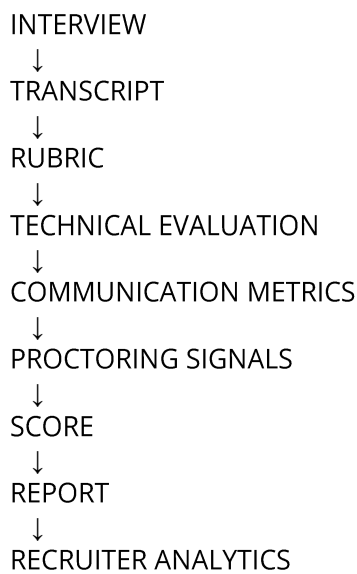


For the core AI interview:





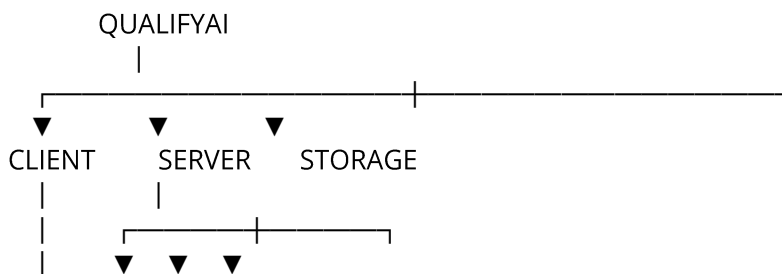
For evaluation:

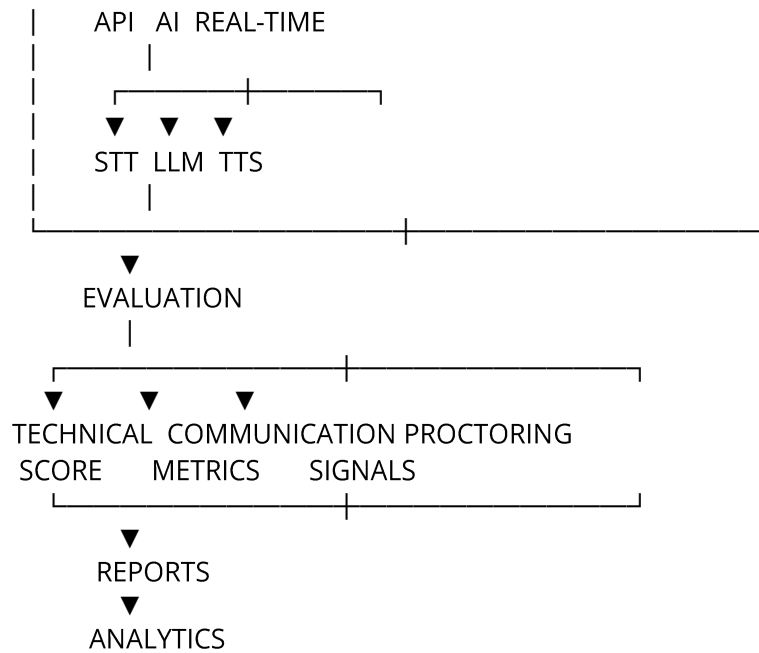


54. Final Definition

QualifyAI Software Architecture is a modular, multi-tenant SaaS architecture that separates the client experience, server application logic, real-time interview infrastructure, AI integrations, evaluation, proctoring, persistent data, storage, and documentation.

The architecture is centered around:





The architecture therefore provides a clear separation between **user experience, application services, AI-powered conversational interviewing, evaluation, assessment integrity, and persistent tenant-scoped data**, while preserving a practical path toward scalable technical recruitment infrastructure.

Document 4 — Software Architecture & Development Structure — Complete.